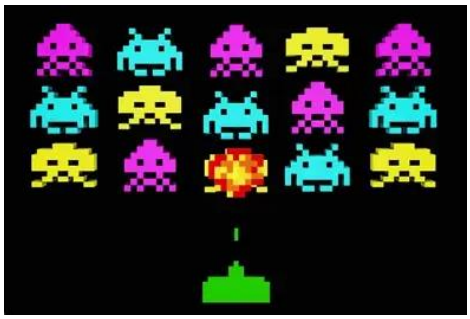
# Shoot me up !



Noé Béran – MID 2b

Vennes

32 périodes

Xavier Carrel

# Table des matières

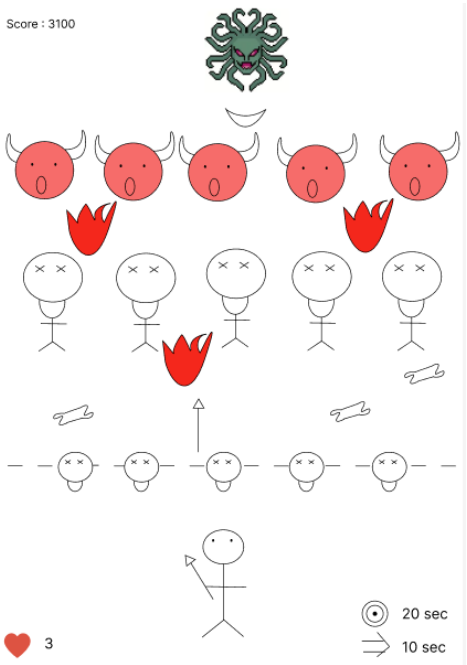
Introduction ..... 3
Vue Globale du jeu ..... 3
Objectif produit ..... 3
Objectif pédagogique ..... 4
Analyse fonctionnelle ..... 4
User Story : ..... 4
1. Joueur ..... 4
2. Tir ..... 4
3. Adversaire ..... 5
4. Attaque adverse ..... 5
5. Défenses ..... 5
6. Score ..... 6
7. Défaite ..... 6
8. Accueil ..... 6
9. Démon ..... 7
10. Squelette ..... 7
11. Meilleur score ..... 7
12. Esquive ..... 7

13. Attaque spéciale ..... 8
14. Medusa ..... 8
Schéma : ..... 8
Planification initiale ..... 10
Rapport de tests ..... 11
Conception ..... 12
Usage de l'IA ..... 17
Bilan du déroulement du projet ..... 18
Bilan produit ..... 19
Conclusion ..... 20

# Introduction

## Vue Globale du jeu

Voici une vue globale de comment seras le jeu avec tout ce qui est dans ce qui est a faire et les idées mais sans les idées profondes




## Objectif produit

Le but du projet est de créer un jeu complet en C# dans le style de Space Invader mais dans le thème de notre choix, pour moi le thème sera la mythologie grecque/les enfers.

Pour cela le joueur sera un gladiateur qui lanceras des lances sur des démons et des squelettes. Si le temps le permet je rajouterais aussi médusa dans le jeu.

### Objectif pédagogique

Le but pédagogique du projet est d'apprendre la programmation orienté objet en C#. Ce projet renforce aussi nos capacités à apprendre à comprendre un cahier des charges, rédiger des users stories et apprendre à utiliser GitHub.

# Analyse fonctionnelle

## User Story :

### 1. Joueur

En tant que joueur je souhaite pouvoir me déplacer de droite a gauche et de haut en bas du terrain pour esquiver les attaques.

Voir schéma Player

*Critère d'acceptation :*

- Le joueur va vers le haut lorsque on appuie sur la touche W
- Le joueur va vers le bas lorsque on appuie sur la touche S
- Le joueur va vers la gauche lorsque on appuie sur la touche A
- Le joueur va vers à droite lorsque on appuie sur la touche D
- Le joueur ne bouge pas lorsque aucune touche n'est appuyée
- Si le joueur arrive au bord du terrain, il est bloqué pour ne pas sortir de l'écran
- Le joueur se déplace à vitesse constante
- Le joueur a 3 vies

### 2. Tir

En tant que joueur je souhaite pouvoir tirer des projectiles afin de tuer les adversaires.

Voir schéma Player

*Critère d'acceptation :*

- Si j'appuie sur la touche espace, un projectile pars du joueur
- Le projectile pars droit devant le joueur a vitesse constante
- Le projectile s'arrête au contact d'une défense
- Le projectile s'arrête au contact d'un adversaire
- Il y a un petit cooldawn entre les tirs
- Les projectiles infligent 1 dégât aux monstres

## 3. Adversaire

En tant que joueur je souhaite avoir une vague de plusieurs adversaires m'attaquant afin d'avoir de la difficulté

Voir schéma monster

*Critère d'acceptation :*

- Il y a au moins 2 lignes de monstres
- Chaque ligne contient au moins 5 monstres
- Les monstres les plus forts sont derrière
- Les monstres se déplacent droit vers le joueur sans faire de virage
- Les monstres se déplacent lentement
- Si un monstre arrive au niveau des obstacle, il fait des dégâts au obstacles jusqu'à que le joueur le tue
- Si un monstre est au niveau des obstacles alors qu'il n'y en reste plus du tout, le joueur a perdu

## 4. Attaque adverse

En tant que joueur je souhaite que les monstres me tirent dessus afin de rajouter de la difficulté.

Voir schéma monster

*Critère d'acceptation :*

- Les projectiles se déplacent de manière linéaire
- Les projectiles sont rapides
- Les projectiles sont différents en fonction du monstre
- Si les projectiles touchent un projectile du joueur, les 2 disparaissent
- Les projectiles infligent 1 dégât au joueur

## 5. Défenses

En tant que joueur je souhaite avoir au moins 5 défenses destructibles afin de pouvoir me protéger des attaques ennemies.

Voir schéma defense

*Critère d'acceptation :*

- 5 défenses sont devant le joueur
- Les défenses bloquent les tirs
- Au bout de 3 tirs arrêté, la defense disparait

- Le score diminue de 200 lorsque le joueur le casse

## 6. Score

En tant que joueur je souhaite avoir un score d'affiché afin de suivre mon niveau au fils des parties.

Voir schéma score

*Critère d'acceptation :*

- Le score est constamment affiché en haut de l'écran
- Le score commence a 0
- Le score augmente différemment en fonction du monstre tué
- Le score diminue si le joueur casse une défense

## 7. Défaite

En tant que joueur je souhaite avoir un écran différent lors d'une défaite afin de savoir lors que j'ai perdu.
Voir schéma lose

*Critère d'acceptation :*

- L'écran s'affiche automatiquement lorsque le joueur n'a plus de vie
- Il y a un bouton pour rejouer
- Il y a un bouton qui fait revenir au menu
- Cela indique le score du joueur

## 8. Accueil

En tant que joueur je souhaite avoir un écran d'accueil en lancent le jeu afin de pouvoir choisir quand lancer la partie et regarder les touches.

Voir schéma lobby

*Critère d'acceptation :*

- L'écran s'affiche dès le lancement du jeu
- On peut lancer la partie à partir de la
- On peut voir les touches
- On peut quitter le jeu

## 9. Démon

En tant que joueur je souhaite qu'un monstre sois un démon qui tanque 2 coups et qui lance des boules de feu afin de diversifier les adversaires
Voir schéma Monster

*Critère d'acceptation :*

- Il peut se prendre 2 attaques avant de mourir
- Il tir des boules de feu
- Il est placé derrières les squelettes
- Il donne 300 points à sa mort

## 10. Squelette

En tant que joueur je souhaite qu'un monstre sois un squelette qui tanque 1 coups et qui lance des os afin de diversifier les adversaires.
Voir schéma Monster

*Critère d'acceptation :*

- Il meure au premier projectile du joueur qui le touche
- Il lance des os
- Il est placé devant les démons
- Il donne 150 points à sa mort

## 11. Meilleur score

En tant que joueur je souhaite avoir un meilleur score de toute mes parties afin de pouvoir me challenger à le dépasser.
Voir schéma lose

*Critère d'acceptation :*

- Le meilleur score est affiché à la fin de la partie
- Si le score est plus haut que le meilleur score, le meilleur score devient le score
- Le meilleur score est enregistré entre les parties

## 12. Esquive

En tant que joueur je souhaite avoir une esquive qui se recharge avec le temps afin de pouvoir esquiver les projectiles les plus dors.
Voir schéma player

*Critère d'acceptation :*

- Le joueur fait une esquive quand la touche « shift » est appuyée
- Lors de l'esquive, le joueur est invincible

- Lors de l’esquive, le joueur se déplace très vite sur une petite distance
- L’esquive peut s’utiliser rarement (15 secondes de chargement)

### 13. Attaque spéciale

En tant que joueur je souhaite avoir une attaque spéciale de temps en temps afin de tuer les monstres plus facilement.

Voir schéma player

*Critère d’acceptation :*

- Le joueur fait l’attaque spéciale quand la touche « e » est appuyée
- Lors de l’attaque spéciale, le joueur lance un gros bouclier
- Le bouclier tue 2 ennemis, peu importe leurs PV avant de se casser
- Le bouclier casse entièrement une défense avant de se casser
- Le bouclier passe en travers des attaques adverse

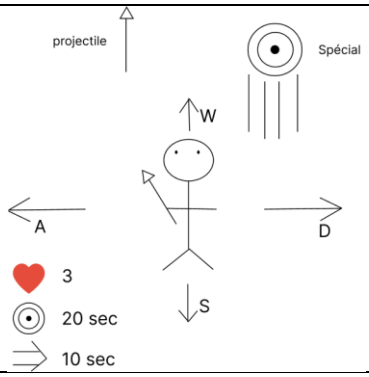
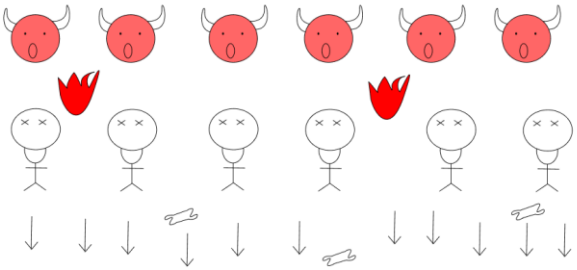
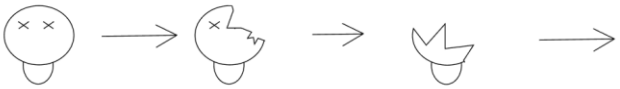
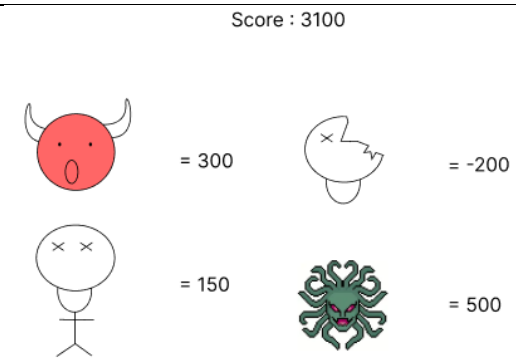
### 14. Medusa

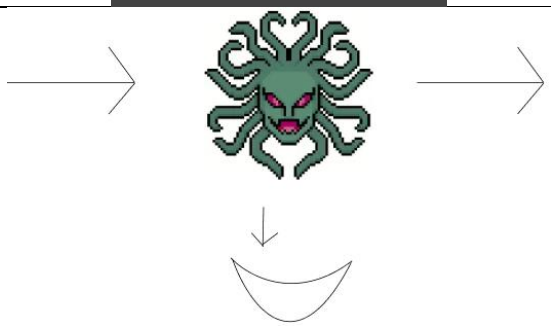
En tant que joueur je souhaite qu’un monstre soit médusa qui passe de temps en temps afin de complexifier le jeu.
Voir schéma medusa

*Critère d’acceptation :*

- Medusa apparait en haut à gauche de l’écran et sort en haut à droite
- Medusa n’as pas besoin d’être tuée pour gagner la vague
- Medusa lance un projectile qui immobilise le joueur lorsqu’elle arrive au milieu de l’écran
- Elle donne 500 points si tuée
- Elle apparait 2x pas vague maximum
- Elle apparait à des moments aléatoires

## Schéma :

| | |
|---|---|
| Player |  |
| Monster |  |
| Defense |  |
| Score |  |
| Lose |  |

| | |
|---|---|
| Lobby |  |
| Medusa |  |

## Planification initiale

| User Story | Date de finition de la US |
|---|---|
| 1 – Mouvement du joueur | 11.09.2026 |
| 2 – Tir | 11.09.2026 |
| 3 - adversaire | 18.09.2026 |
| 23 – démon | 18.09.2026 |
| 24 - squelette | 18.09.2026 |
| 4 – attaque adverse | 18.09.2026 |
| 5 - défenses | 25.09.2026 |
| 7 – score | 25.09.2026 |
| 8 - Défaite | 02.10.2026 |
| 9 - Victoire | 02.10.2026 |
| 17 - Accueil | 02.10.2026 |
| 10 – Meilleur score | 09.10.2026 |
| 11 - esquive | 09.10.2026 |
| 15 – attaque spéciale | 16.10.2026 |
| 25 - medusa | 23.10.2026 |
| 12 - cerbère | 30.10.2026 |

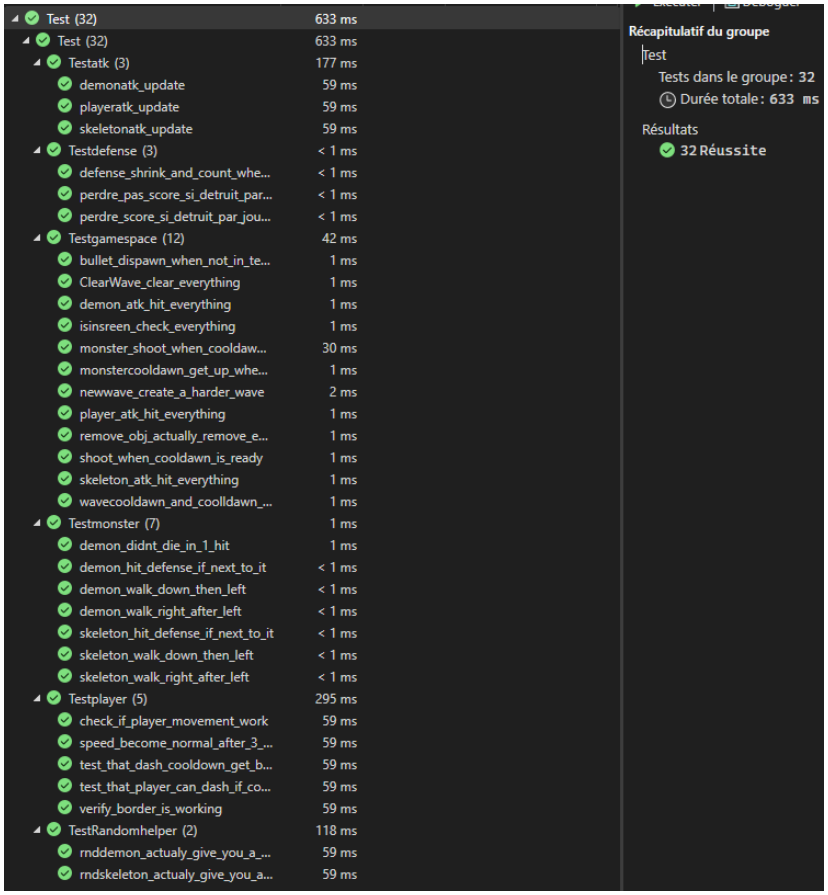
# Rapport de tests

Les tests ont été faits sur MStest avec plusieurs tests par fonction qui change des variables. Il y a des tests quasiment partout. Voici la liste des tests et ce qu'ils vérifient:

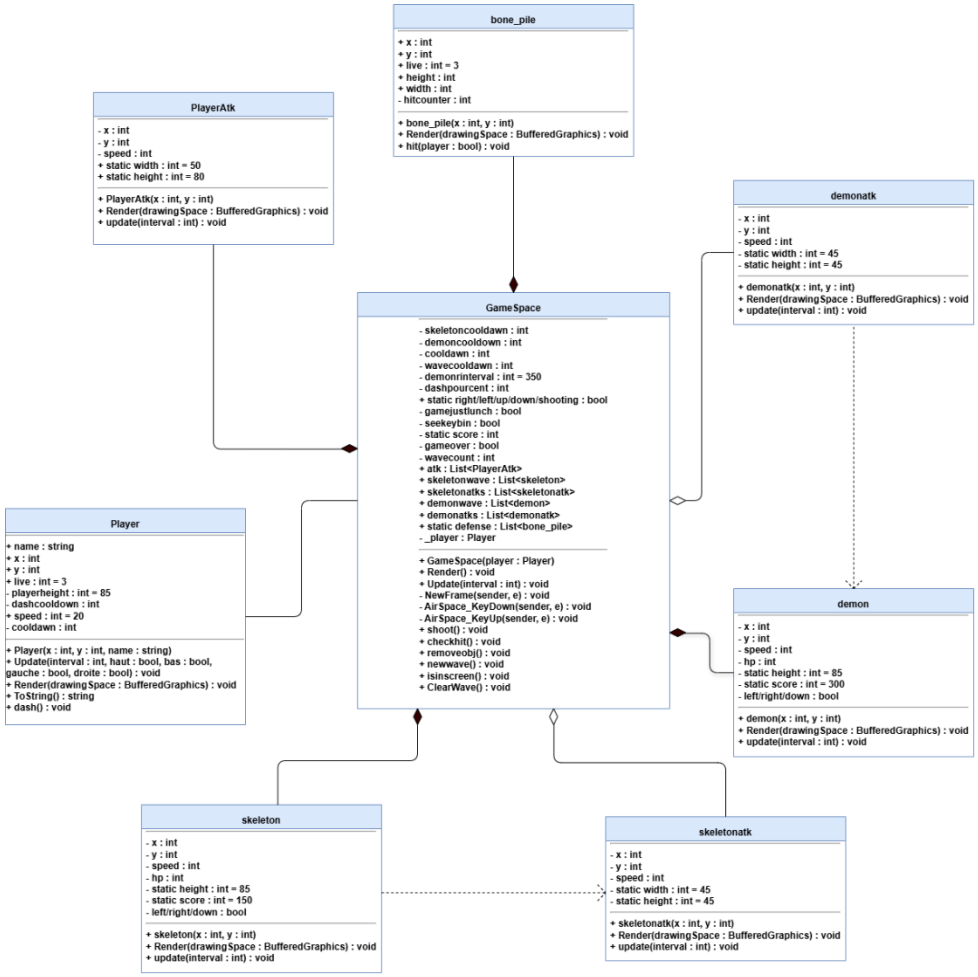
| Test réalisé | Ce que cela vérifie |
|---|---|
| skeletonatk_update | Vérifie que les attaques des squelettes s'updatent correctement et avancent |
| demonatk_update | Vérifie que les attaques des démons s'updatent correctement et avancent |
| playeratk_update | Vérifie que les attaques du joueur s'updatent correctement et avancent |
| defense_shrink_and_count_when_hit | Vérifie que la défense contabilise les coups et rapetisse de la bonne taille |
| perdre_score_si_detruit_par_joueur | Vérifie que le joueur perde du score s'il détruit lui-même la défense |
| perdre_pas_score_si_detruit_par_monstre | Vérifie que le joueur ne perde pas de score s'il ne détruit pas lui-même la défense |
| rndskeleton_actualy_give_you_a_skeleton_in_the_liste | Vérifie que la fonction rndskeleton donne un squelette qui fait partie de la liste que on lui donne |
| rnddemon_actualy_give_you_a_skeleton_in_the_liste | Vérifie que la fonction rnddemon donne un démon qui fait partie de la liste que on lui donne |
| skeleton_hit_defense_if_next_to_it | Vérifie que si un squelette arrive en bas de la limite et qu'il reste des défenses, il commence à les casser |
| demon_hit_defense_if_next_to_it | Vérifie que si un démon arrive en bas de la limite et qu'il reste des défenses, il commence à les casser |
| skeleton_walk_down_then_left | Vérifie que le squelette tourne à gauche au début après être descendu |
| demon_walk_down_then_left | Vérifie que le démon tourne à gauche au début après être descendu |
| skeleton_walk_right_after_left | Vérifie qu'après être tourné à gauche, le squelette tourneras à droite lors du prochain virage |
| demon_walk_right_after_left | Vérifie qu'après être tourné à gauche, le démon tourneras à droite lors du prochain virage |
| demon_didnt_die_in_1_hit | Vérifie que le démon puisse prendre 1 coups sans mourir |
| check_if_player_movement_work | Vérifie si le joueur se déplace correctement dans toutes les directions |

| | |
|---|---|
| verify_border_is_working | Vérifie que si un joueur sort de 1 des 4 cotés de la map, il est remis sur la map |
| test_that_dash_cooldown_get_bigger _if_is_lower_than_100 | Vérifie que dashcooldown augmente s'il est en dessous de 100 et que sinon il ne fait rien |
| test_that_player_can_dash_if_cooldown _is_over_100_and_cant_if_lower | Vérifie que le joueur augmente de vitesse s'il dash avec le cooldown prêt et que sinon il garde sa vitesse |
| speed_become_normal_after_3_frames | Vérifie que le dash dure exactement 3 frames avant de revenir à la normal |
| wavecooldawn_and_coolldawn_do_plus_1_when_no_monster | Vérifie que e cooldown de la prochaine vague de monstre augmente s'il y en a plus et que le cooldown du joueur augmente aussi |
| monstercooldawn_get_up_when_ther_is_monster | Vérifie que le cooldown des tirs ennemis augmente en fonction du nombre d'ennemis |
| monster_shoot_when_cooldawn_is_ready | Vérifie que les ennemis tirs et réduisent leurs cooldown s'il est prêt |
| shoot_when_cooldawn_is_ready | Vérifie que le joueur puisse tirer et remettre son cooldown a 0 s'il est prêt |
| bullet_dispawn_when_not_in_terrain | Vérifie que toutes les attaques disparaissent si elles sortent de l'airspace |
| player_atk_hit_everything | Vérifie les collisions entre les tirs du joueur et tout le reste |
| skeleton_atk_hit_everything | Vérifie les collisions entre les tirs du squelette et tout le reste |
| demon_atk_hit_everything | Vérifie les collisions entre les tirs du démon et tout le reste |
| remove_obj_actually_remove_everything | Vérifie si la fonction removeobj supprime bien ce que on lui demande |
| newwave_create_a_harder_wave | Vérifie que demonwave crée bien une nouvelle vague avec + de monstre que la vague d'avant |
| ClearWave_clear_everything | Vérifie que la fonction clearwave supprime bien tous les monstres |
| isinsreen_check_everything | Vérifie que la fonction isinscreen regarde bien toutes les instances de tous les objets et tous les coté de l'écran |

Résultat :


Test (32) 633 ms
Test (32) 633 ms
Testatk (3) 177 ms
demonatk_update 59 ms
playeratk_update 59 ms
skeletonatk_update 59 ms
Testdefense (3) < 1 ms
defense_shrink_and_count_whe... < 1 ms
perdre_pas_score_si_detruit_par... < 1 ms
perdre_score_si_detruit_par_jou... < 1 ms
Testgamespace (12) 42 ms
bullet_dispawn_when_not_in_te... 1 ms
ClearWave_clear_everything 1 ms
demon_atk_hit_everything 1 ms
isinsreen_check_everything 1 ms
monster_shoot_when_cooldaw... 30 ms
monstercooldawn_get_up_whe... 1 ms
newwave_create_a_harder_wave 2 ms
player_atk_hit_everything 1 ms
remove_obj_actually_remove_e... 1 ms
shoot_when_cooldawn_is_ready 1 ms
skeleton_atk_hit_everything 1 ms
wavecooldawn_and_coolldawn_... 1 ms
Testmonster (7) 1 ms
demon_didnt_die_in_1_hit 1 ms
demon_hit_defense_if_next_to_it < 1 ms
demon_walk_down_then_left < 1 ms
demon_walk_right_after_left < 1 ms
skeleton_hit_defense_if_next_to_it < 1 ms
skeleton_walk_down_then_left < 1 ms
skeleton_walk_right_after_left < 1 ms
Testplayer (5) 295 ms
check_if_player_movement_work 59 ms
speed_become_normal_after_3_... 59 ms
test_that_dash_cooldown_get_b... 59 ms
test_that_player_can_dash_if_co... 59 ms
verify_border_is_working 59 ms
TestRandomhelper (2) 118 ms
rnddemon_actualy_give_you_a_... 59 ms
rndskeleton_actualy_give_you_a... 59 ms
Récapitulatif du groupe
Test
Tests dans le groupe : 32
Durée totale : 633 ms
Résultats
32 Réussite

# Conception




## GameSpace

C’est la partie principale de mon code, ses variables sons :

- Skeletoncooldawn, qui regarde quand le squelette peut tirer
- Demoncooldown, qui regarde quand le demon peut tirer
- Cooldawn, qui regarde quand le joueur peut tirer
- Wavecooldawn, qui regarde quand une nouvelle vague doit apparaitre
- Up/down/left/right, qui servant a savoir ou le joueur doit se déplacer
- Gamejustlunch, seekeybin et gameover, qui servant a savoir quel écran afficher
- Score, qui contient le score du joueur
- Wavecount, qui contient le numéro de la vague
- Toutes les listes des monstres, attaques, défenses et ce qu’il faut delete
- Le joueur

Ainsi que ses fonctions :

- GameSpace, le constructeur de la classe
- Render, qui affiche tous les éléments
- Update, qui met à jour la position de tous les éléments
- NewFrame, qui fait appel les fonctions nécessaires a une nouvelle image
- Shoot, qui crée une nouvelle balle a la position du joueur
- Checkhit, qui vérifie toutes les collisions
- Removeobj, qui supprime tous les objets qui doivent l'être
- Newwave, qui crée une nouvelle vague d'ennemis
- Isinscreen, qui vérifie que tous sont bien dans l'écran
- Clearwave, qui supprime tous les ennemis

Gamespace est la base du code, c'est elle qui contient tout le reste et coordonne, dit ce qu'il faut faire

## Player :

C'est tout ce qui concerne le joueur, ses variables sont :

- Name, qui contient le nom du joueur
- X et Y, qui contienne la position du joueur sur l'airspace
- Live, qui contient le nombre de vie du joueur
- Playerheight, qui contient les dimensions du joueur
- Dashcooldown, qui contient le cooldown avant le prochain dash
- Speed, qui est la vitesse du joueur lors du jeu
- Cooldown, qui est le cooldown avant de repouvoir tirer

Ainsi que ses fonctions :

- Player, le constructeur de la classe
- Update, qui met a jour la position du joueur en fonction de la direction
- Render, qui affiche le joueur sur l'airspace
- Dash, qui augmente vivement la vitesse du joueur

## PlayerAtk :

C'est tout ce qui concerne les attaques du joueur, ses variables sont :

- X et y, pour la position du tir

- Speed, pour la vitesse de la balle
- Width et height, pour les dimensions de la balle

Ainsi que ses fonctions :

- PlayerAtk, le constructeur de la classe
- Render, qui affiche le tir sur l'airspace
- Update, qui met a jour la position du tir

## Skeleton :

C'est le monstre le plus faible du jeu, ses variables sont :

- X, y , speed, height, pour les memes raisons que dans les autres classes
- Score, qui est le score que le joueur gagne en tuant un squelette
- Left, right, down, qui dit  dans quel direction vas le squelette

Ainsi que ses fonctions

- Skeleton, le constructeur de la classe
- Render et update, pour les mêmes raisons que les autres classes

## Skeletonatk

C'est l'attaque du squelette, ses variables sont :

- X, y, speed, width, height

Ainsi que ses fonctions :

- Skeletonatk, le constructeur de la classe
- Render, update

## Demon

C'est le monstre le plus resistant , voici ses variables :

- X, y, speed, height
- Hp, le nombre de vie qu'il a
- Score, qui est le score que le joueur gagne en tuant un demon

- Left, right, down, qui dit  dans quel direction vas le demon

Ainsi que ses fonctions :

- Demon, le constructeur de la classe
- Render, update

### demonatk

C'est l'attaque du demon, ses variables sont :

- X, y, speed, width, height

Ainsi que ses fonctions :

- demonnatk, le constructeur de la classe
- Render, update

Demonatk et skeletonatk ont une dette technique et je peux créer une interface qui ralie les 2, pour minimiser le code

### Bone_pile

Ce sont les défenses du jeu, ses variables sont :

- X, y, height, width
- Hitcounter, qui contient le nombre de fois que la défense a été touchée

Ainsi que ses fonctions :

- Bone_pile, le constructeur
- Render
- Hit, qui contabilise un coup

## Usage de l'IA

La seul IA que j'ai utilisée est Claude, je l'ai utilisé pour m'expliquer certaines erreurs dans mon code que je n'arrivais pas à résoudre. Généralement, je comprenais ce que j'avais fait de faut assez rapidement comme ça mais sinon, je prenais bien mon temps pour comprendre l'erreur, comment la corriger, pourquoi ça marche etc.

Après chaque réponses de Claude, je regardais comment il s'y prenait pour corriger mon erreur dans le code et, si je ne comprends pas une ligne, lui demande de m'expliquer ou alors je change cette ligne par quelques chose que je connais.

Si les réponses étaient mal faites, ne fonctionnait pas, ou me paraissait trop compliquées, je lui demandait de reformuler, ou alors de trouver une autre solution si possible

Exemple (reconstitution car le chat a été supprimé):

Lors du changement du nom des fichier en shoot me up, plus aucun de mes using ne fonctionnait, pourtant, les namespaces étaient les bons et using marchaient avec, mais pourtant le programme ne les retrouvait pas : j'ai donc expliqué la situation a Claude et il m'as expliqué que vu que j'avais changé le nom du fichier, je devais changer les dépendances du projet pour les remettre à jour.

Le problème a été résolu du premier coup, cette discussion m'as appris penser a mettre a jour les dépendances des fichiers si un gros problème comme celui-ci apparait

J'ai aussi utilisé Claude pour m'expliquer des concepts que je ne comprenais pas/ne connaissait pas, comme les diagrammes de classes, qui n'avaient pas été expliqué ou alors comment créer des tests unitaires, car je n'avais pas tout compris au cours.

Je ne voulais pas beaucoup utiliser l'IA pour écrire du code alors je l'ai gardée pour les moments ou j'était réellement bloqué et que je ne trouvais pas la réponse sur internet. Elle m'as surtout accéléré des recherche sur des erreurs ou des concepts que je ne connaissait pas.

## Bilan du déroulement du projet

Lors de la planification initiale, j'ai été bien trop optimiste, j'ai voulu mettre toutes mes idées alors que je n'avais pas le temps et je ne savais pas combien de temps prend les choses a être faites et j'avais oublié de prendre en compte le fait qu'on devait faire le rapport, les tests qu'il y avait des vacances et que on allait pas commencer à la semaine 1 a coder :

| User story | Date de fin prévue | Vraie date de fin |
|---|---|---|
| 1 – Mouvement du joueur | 11.09.2026 | 18.09.2026 |
| 2 – Tir | 11.09.2026 | 25.09.2026 |
| 3 - adversaire | 18.09.2026 | 02.10.2026 |
| 23 – démon | 18.09.2026 | 02.10.2026 |
| 24 - squelette | 18.09.2026 | 02.10.2026 |
| 4 – attaque adverse | 18.09.2026 | 02.10.2026 |
| 5 - défenses | 25.09.2026 | 09.10.2026 |

| | | |
|---|---|---|
| 7 – score | 25.09.2026 | 02.10.2026 |
| 8 - Défaite | 02.10.2026 | 26.10.2026 |
| 9 - Victoire | 02.10.2026 | Pas fait |
| 17 - Accueil | 02.10.2026 | 28.10.2026 |
| 10 – Meilleur score | 09.10.2026 | Pas fait |
| 11 - esquive | 09.10.2026 | 29.10.2026 |
| 15 – attaque spéciale | 16.10.2026 | Pas fait |
| 25 - medusa | 23.10.2026 | Pas fait |
| 12 - cerbère | 30.10.2026 | Pas fait |

Les principales causes de mon retard dans les US sont :

- Un retard pris dès le début car je pensais que je coderais dès la semaine 1
- Les vacances durant lesquels je n'ai peu/pas travaillé
- Plusieurs erreurs qui m'on fait perdre beaucoup de temps, ex : le mouvement du joueur qui fonctionnait que a moitié et qui m'as pris une demi-journée a débugger

Pour travailler, j'ai utilisé github desktop, qui est beaucoup plus simple que celui en ligne, ce qui m'as fait gagner du temps. Je n'ai pas utilisé le GitHub journal pour faire le jdt car j'ai l'habitude de le faire sur Excel et cela me semblait bien.

# Bilan produit

## US réalisés

Toutes les US aubligatoires :

- Les mouvements
- L'attaque du joueur
- Les 2 types d'ennemis
- Les attaques adverses
- Les défenses
- Le score
- Le système de défaites

Ont été réalisées dès le début pour être sûr de ne pas arriver a la fin du projet et qu'il me manque quelque chose d'indispensable

et quelques US en plus, comme l'écran d'accueil et l'esquive ont été ajouté avec le temps restant.

Les US non-réalisées sons des ajout « bonus », dispensables, au jeu que je n'ai pas eu le temps de faire, comme par Example, une attaque spéciale au joueur.

## Changement dans les US

- Joueur :
    - J'ai changé les touches de déplacement sur les flèches directionnelles au lieu de WASD car au ressenti en jouant, je trouve plus simple
- Écran d'accueil :
    - J'ai enlevé la fonctionnalité de pouvoir quitter le jeu depuis cet écran car cela ne me semblait pas intéressent de l'avoir
- Esquive :
    - La touche est E et non shift car shift me semblait bien si le déplacement était sur WASD mais vu que je l'ai changé, j'ai changé celle-là aussi
    - Le joueur n'est pas invincible car cela est trop fort s'il l'est
    - Le temps d'attente pour charger l'esquive est passé de 15S a 10S car dans le jeu, 15S c'est vraiment beaucoup et le joueur ne peut quasiment jamais utiliser son esquive
- Défaite :
    - Il n'y a pas d'option pour revenir au menu car je trouve que le menu n'ajoute rien d'intéressent à y retourner après une partie
- Squelette/démon :
    - Ce sont les démons qui sont devant car j'estime que vu qu'ils ont 2 vies, ils encaisseraient les dégâts pour les squelettes
- Défense :
    - J'ai augmenté leurs nombres de vie car sinon elles cassaient trop vite

# Conclusion

En conclusion, je trouve que le projet était vraiment bien, on avait des libertés dans la manière de faire et de quoi faire.

Si je dois refaire se projet une foi, comme amélioration, j'essayerais de mieux ordonner mon code pour éviter les répétitions et la dette technique (en TODO dans le code), car j'ai passé beaucoup de temps, à la fin du projet, a refactoriser mon code car je m'étais mal organisé et potentiellement rajouter des fonctionnalités supplémentaires.

Ce projet m'as appris toute la base de la programmation orienté objets, a créer des tests d'acceptations avec les US et il m'as beaucoup plus familiariser avec GitHub, que je commence a vraiment bien maitriser